

Probabilistic Indoor Person Search for Mobile Robots

Andri Wild

Institute for Mobile and Distributed Systems

FHNW

Windisch, Switzerland

andri.wild@students.fhnw.ch

Abstract—A mobile robot has to find a certain person in a building. The robot does not know their movement profile and must find them before an appointment starts. We present a two-level approach for this task, a site tour over the rooms of the building and a room tour inside a single room. For the site tour the robot estimates a beta-smoothed presence probability for every room from its own observations. We state the choice of rooms under the time budget until the appointment as an orienteering problem. A greedy method sorts the rooms by reward per time cost. The room tour is precomputed and covers the room completely. We evaluate the approach in a discrete event system that models the movement of people as a semi-Markov chain, over 30 simulation days and 20 runs per variant. To find the person in 90 percent of the search jobs, the robot has to start 14 minutes before the appointment. Several levers lead to this goal. The time budget until the appointment matters most. The smoothed estimate, the prior knowledge and a larger detection range all make up for the same lack of observations.

Index Terms—Discrete event simulation, Markov chains, Mobile robots, Orienteering problem, Person search, VisLib2

I. INTRODUCTION

Mobile autonomous robots appear more and more in our daily life and take over many different tasks. One task that is still rare but nevertheless important is the search for a person. The search brings special challenges. People behave in an unexpected way and do not want to be watched all the time. A human solves this task with assumptions and observations that are hard to put into an algorithm. We present a two-level approach for it and show how an autonomous robot can find a person within a given time budget.

To develop such algorithms we need a simulation. We use a discrete event system [1], because it computes results over long periods quickly.

The test building is a single floor. The robot moves freely on this floor. We have left out the doors, so every room is open. Figure 1 shows the floor plan. The color of a room gives its type. The red dot marks the entry position through which the robot enters the room. Note the outlines of the single rooms. We see that they are not simple rectangles. Room geometry and furniture make them concave, and we model both as part of the outline. They hide parts of a room and therefore decide how many viewpoints the robot needs for a complete search.

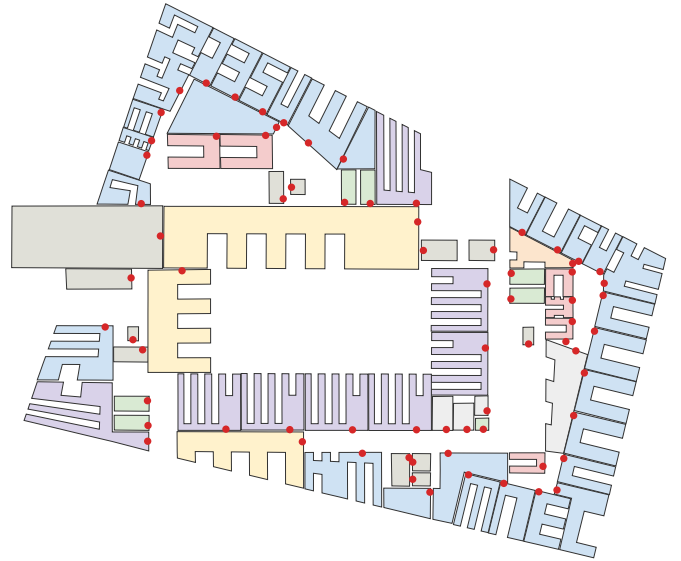


Figure 1. Floor plan of the test building, colored by room type, with red dots as entry positions. Most room outlines are not simple rectangles, room geometry and furniture make them concave.

Note also the middle of the building. We see a large inner courtyard there, with the offices around it along the outer facade. The robot has to drive around this courtyard. Two rooms on opposite sides are therefore far apart. For the site tour in Section IV.B we compute with real driving paths for this reason.

During a simulation run, a fixed number of people stay in the building. Every person has an own movement profile. We model the movement profiles with a transition matrix, introduced in Section III.A.

The job is done by a robot that is an autonomous mobile platform. It has a map of the floor and drives on this map by itself to a given room. Cameras give it an all-round view over 360 degrees. With them it perceives people and identifies them, as long as they are close enough. Perceiving works over a longer distance than identifying.

Every search job has a deadline. The robot has to find the person before it, and the span until then is the time budget of the search.

For the person search we present a two-level approach. The upper level works on the building level and decides which rooms the robot visits and in which order. We call it

the *site tour*. The lower level drives through a single room completely. We call it the *room tour*.

Two questions guide this work. If the robot has to find a person with a probability of 90 percent, how much time does it have to plan for the search? Which factors influence the duration of the search? We first set the boundary conditions, how much time a search needs and how far the robot has to see. On this basis we study three measures. These are the use of its own observations, the prior knowledge that the robot has before its first observation and the ordering of the rooms by cost.

We build directly on several earlier works. The background on discrete event systems and semi-Markov chains comes from Cassandras and Lafortune [1]. We compute the driving distances with the path planner of the navigation stack Nav2 [2], [3]. The visibility points of a room come from the library VisLib2 [4].

II. RELATED WORK

The search for a person in a building is usually stated as a planning problem on a graph. The nodes are rooms or areas, the edges are the driving times between them. Every node carries a probability that the wanted person is there. We look for the order of nodes that gives the shortest expected time until the person is found.

Krajník et al. showed that this probability should not be treated as a constant [5]. People follow daily and weekly routines, so the presence in a room depends on the time of day. They modeled this behavior with frequency spectra and with periodic Gaussian mixtures and learned the models from observations over several weeks. They computed the search path exactly with branch and bound. Compared to a static model, the mean search time dropped by about 25 percent and the median by 35 to 65 percent.

Three limits of this approach matter for us. First, the wanted person stays in place during the search. Second, a node is an area without inner geometry. A person therefore counts as found as soon as the robot reaches the node. How a room is searched is not part of the model. Third, the method is exact and therefore limited in size. For 25 nodes the authors already report a computing time of 60 seconds.

Mehdi and Berns followed a similar approach on the level of a flat [6]. They determined a set of locations with a field of view as large as possible and rated them with a combination of presence probability and navigation cost. They stated the selection as a Markov decision process, where the probability serves as reward and the navigation cost enters as its inverse. With a similar probability the closer location therefore wins. The probabilities have an hourly resolution and are updated after a successful search.

Their model also leaves the search inside a room open. The locations see an area as large as possible, but a complete coverage is not guaranteed. The authors state themselves that the robot can miss the person although it is in the right room. Their evaluation also took place in a flat with four rooms and five locations. The question which of many rooms the robot chooses does not arise there.

For the search inside a single area Sarmiento et al. propose a two-stage method [7]. First they choose a set of locations whose visibility regions together cover the whole polygon. These locations are the guards from the art gallery problem. Then they look for the order in which the robot visits them, so that the mean time until the object is found gets as short as possible. The ordering is NP-hard, which is why the authors use a greedy heuristic. It rates the next location by the probability of finding the object there divided by the travel time to get there. Our score in (3) follows the same idea.

Two things are different in our case. Their object does not move and lies with equal probability at any point of the polygon, so the robot has nothing to learn from earlier visits. The method also works on a single polygon and does not choose between the rooms of a building.

The selection under a limited budget matches the orienteering problem, which Golden et al. introduced [8]. Every node has a score. We look for the path that maximises the collected score without going over the budget. The problem is NP-hard, which is why heuristics are used in practice [9]. In addition, the score of a node and the time to reach it are independent of each other and often work against each other. A common approach therefore rates the nodes by the ratio of score to distance.

Each of these works covers one of the two levels. Krajník et al. as well as Mehdi and Berns decide where the robot drives. How it searches a place once it has arrived is not part of their models. Sarmiento et al. do the opposite. They search one area completely, but which area to search is not part of their work. We do both. The robot chooses the rooms and then searches each of them completely. Our scenario also differs in two points. The wanted person keeps moving during the search and is on the way between two rooms from time to time. In addition, there is the time budget of the site tour, so the search cannot run as long as it wants.

III. BACKGROUND

A. Markov Chains

The basis for the simulation of people is the Markov chain. Its state space is the finite set of all states that a person can take. A state is a room in our case. A person is in exactly one room at any point in time. State space and transition probabilities define the chain. We define one chain per person [1, p. 406]. The chain has no memory, so the next step depends only on the current state [1, p. 406].

The transition matrix P in (1) collects these probabilities. Row i gives with which probability a person changes from room i into each of the other rooms.

$$P = \begin{pmatrix} p_{11} & p_{12} & \cdots & p_{1n} \\ p_{21} & p_{22} & \cdots & p_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{n1} & p_{n2} & \cdots & p_{nn} \end{pmatrix} \quad (1)$$

Every entry lies between zero and one, and the entries of a row add up to one [1, p. 410].

B. Time Dependence

The Markov chain changes state in fixed steps. The stay time is therefore not free, it follows from the chain itself and has a geometric distribution in the discrete-time case [1, p. 413]. That is too restrictive, because different rooms have very different stay times.

For this reason, we use a semi-Markov chain [1, Sec. 6.5]. With it the stay time follows an own distribution. The change of state still fulfills the Markov property, while we model the time in between freely. The sampled stay time sets the point in time of the room change. We call this point in time a transition event.

The stay time depends on the room. We therefore assign every room a room type, which sets the distribution and its parameters. The transition matrix still gives the target room, but the stay time is sampled separately from an own distribution per room. The diagonal entry p_{ii} stands for staying in room i , which the stay time now decides. We therefore set the diagonal of the transition matrix to zero, and every transition leads into another room.

C. Discrete Event System

A discrete event system has a discrete state space. The state changes only when an event happens and stays the same between two events [1, p. 30]. The simulation time therefore does not run in fixed steps, it jumps from one event time to the next.

One simulation run is a sample path, that is the timed sequence of events $(e_1, t_1), (e_2, t_2), \dots$ from a given initial state [1, p. 33]. We have implemented this with the event scheduling scheme [1, p. 319].

Figure 2 shows the event scheduling scheme. In the middle there is the event list with the pending events and their times. Note the two arrows that lead from the first list entry to the left and to the right. We see that both start at the same entry and that one of them sets the state while the other sets the simulation time. In the model the time therefore does not run on its own, it follows the event list.

Six events are essential for the search. A person changes the room, a robot drive starts, a robot drive ends, the robot scans its surroundings at a viewpoint, a room tour finishes and the robot finds the wanted person. The semi-Markov chain fits into this scheme directly, because the transition event is exactly one of these events.

D. The Robot

We do not model the navigation of the robot. Instead, we compute the driving time from the length of the driving route and a constant driving speed v . The destination is therefore all that a drive needs. We add noise to the driving time, so that the same route does not always take exactly the same time.

The all-round view of the cameras means that the robot has no viewing direction and perceives equally far in all directions.

The robot has two ranges of perception. Within the identification range r_i the robot recognizes which person it is. Within the detection range r_e it only notices that someone is

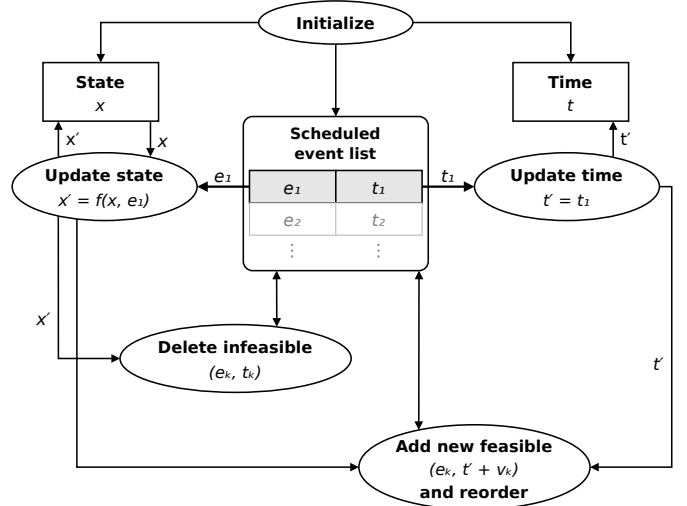


Figure 2. The event scheduling scheme. The earliest entry of the event list sets both the state and the simulation time [1, p. 319].

there. Here $r_e \geq r_i$ always holds. Both ranges are parameters of the scenario.

We assume that the robot already knows all people in the building. If a person is within r_i and no wall breaks the line of sight, then the robot knows who it is. We therefore treat detection and identification as solved.

E. Visibility Coverage and Tour Planning

A room counts as fully searched when the robot has perceived every place in it at least once. A single viewpoint is usually not enough, because the room geometry and the furniture hide parts of the room. We call the area that a robot sees from one viewpoint its visibility polygon. Its size is limited by what the walls hide and by the detection range. We therefore look for a set of viewpoints whose visibility polygons together cover the whole room. We call them visibility points. The room tour thus makes sure that the robot notices every person who is present. Whether it can also identify the person depends on the shorter identification range.

VisLib2 [4] determines these points and their order in one step and therefore delivers a room tour directly. We adopt it without changes. The room tour is not the shortest possible one, but it covers the room completely. Its length gives the cost of a room.

IV. METHOD

The robot knows neither the transition matrix nor the stay times of a person. Instead, it estimates how often the person is in a room. For this it notes at every visit of a room whether it has met the person there. From these hits and misses we get an estimated presence probability per room, which gets more precise with every observation.

Observations do not only arise for the wanted person. When the robot reaches a room, it notes for every person whether it sees them there. Its model therefore grows for all people at the same time.

We do not treat hits and misses in the same way. The robot always keeps a hit, because a person who was seen was in the room for sure. It keeps a miss only if it has searched the room completely. If it leaves the room earlier, it drops the observation, because the person could have been in a part of the room that was not perceived.

A search always follows the same course. The robot gets the job to find a certain person, computes a presence probability for every room from the learned model and plans a site tour from it. It drives to the first room and searches it. If it finds the person, the search is over, otherwise it drives to the next room of the site tour.

A. Estimating the Presence Probability

For every room i the robot counts how often it has met the person there and how often not. Here h_i is the number of hits and n_i is the number of all observations in this room. The simplest variant uses h_i directly as the score.

The naive score has two problems. It counts the hits without setting them against the observations, so a single hit already lifts a room far up. It also ignores the misses, so a room seen once and missed many times outranks every room in which the person was never seen. Rooms without a hit share the same small floor value and are ordered by travel cost alone.

For this reason, we smooth the estimate $\hat{p}_i$ with a prior distribution over the rooms. It adds assumed hits and assumed misses to the real observations, so a room that was never visited still has a defined value and a single hit does not lift it to one at once. The weight of the prior distribution falls with every further observation, and in statistics it matches a beta distribution, which is why we call the estimate beta-smoothed [10, Ch. 3]. As prior knowledge we raise one room per person above all others.

B. Planning the Site Tour

The estimated presence probability holds no information about the distance. A room with a high probability can lie at the other end of the building and cost more time than the gain is worth. The distance therefore comes in one step later.

We state the site tour as an orienteering problem [8]. Every searchable room i is a node with the reward $\hat{p}_i$. The edges are the driving distances in the building. In addition, every node costs time for the room tour, derived from its precomputed length. Large and concave rooms therefore get higher costs.

We exclude the restrooms from the search out of respect for privacy. People still stay there, however, and cannot be found during this time.

We build the site tour with a greedy method. From its position the robot rates every remaining candidate, adds the best one to the site tour and repeats this from the new end point. For the rating we set drive and room tour in relation to the time budget. Here d_i is the driving distance to room i in meters, u_i is the length of its room tour in meters, v is the driving speed in meters per second and $T_{\max}$ is the time budget in seconds.

$$\lambda_i = \frac{d_i + u_i}{vT_{\max}} \quad (2)$$

(2) gives the share of the time budget that a candidate uses. The robot drives the distance d_i to the room and inside it the room tour of length u_i , both with the speed v .

From it we get the score w_i in (3).

$$w_i = \frac{\hat{p}_i}{\lambda_i} \quad (3)$$

A room twice as far away therefore needs a probability twice as high to be equally attractive. The ratio of reward to cost is not new, Sarmiento et al. use it in the same form for their search in a polygon [7].

Before the rating we check every candidate for feasibility. For this we simulate the whole site tour with the added candidate and sum up the time. Candidates for which the time budget until the start of the appointment is no longer enough drop out of the selection. If the robot finds no feasible candidate any more, the planning of the site tour ends.

Because every room costs time, the robot can only search a limited number of rooms. The search therefore does not cover all rooms, it covers as much presence probability as possible within the budget. That is exactly the orienteering problem [9]. If the robot does not find the person in a room, it drives to the next room of the site tour.

C. Driving the Room Tour

Once the robot has reached a room, it drives to its visibility points. If it visits all points, it has perceived every place of the room at least once and the room counts as fully searched. Only then does a miss count for the estimate.

D. Perception at a Distance

On its room tour the robot notices people that it cannot identify from its viewpoint. For the search this is worthless at first, because what counts is which person it is. The robot has to drive closer to identify them.

If it perceives a person it cannot identify at a visibility point, it puts two stops at the front of its room tour, a point close to this person and the point where it stands right now. It stops just inside r_i , that is just close enough to identify the person. Both points lie in the same visibility polygon, so the way is free and the detour only costs driving time. At the first stop the person is within r_i and is therefore identified. After that the robot continues the room tour. The detour happens at most once per person and job.

If the identified person is not the wanted one, an observation for Section IV.A arises.

V. IMPLEMENTATION OF THE SIMULATION

A. Realization

We have implemented the simulation as a discrete event system in C++.

Every room carries a room type. The type sets how long people stay in the room.

Every room has a polygon and exactly one entry position. The robot therefore always enters a room at the same point, no matter where it comes from.

We also compute the distances in advance. For every pair of these positions we determine the path with the Nav2 path planner [2], [3] and store its length in a symmetric matrix, which we load at the start. A distance query at run time is therefore a matrix access and not a path planning. Robot and people use the same matrix and differ only in their speed.

a) State of the Robot:

We model a drive without acceleration. It consists of a start event and an end event, and their distance in time follows from driving distance and v . On top of that we add a delay. Wheel slip, corrections of the low-level controller and small unexpected obstacles all make a drive slower, so the driving time is skewed to the right. We therefore model the delay with a lognormal distribution. Its median is 5 percent of the free driving time, its spread σ is 0.4.

We keep the driving speed v at 1.0 meters per second over all runs.

b) Population of People:

We create the people in advance from the building model. Every person has one room in which they usually stay, and we call it their workplace. The other rooms of a person lie in the same sector of the floor, and in addition there is the kitchen for lunch, which exists only once on the floor.

All people share one transition matrix. What differs between two of them is the state space, that is which rooms the matrix refers to.

c) Movement of People:

We model the movement of a person as a chain of events. When a person arrives in a room, we sample a stay time and put a transition event at this point in time. There the transition matrix gives the target room. However, the person does not arrive there at once. For the duration of the way they are in the state `IN_TRANSIT` and therefore in no room.

On arrival we also sample where in the room the person stands. The point is uniformly distributed over the area of the room polygon and stays the same until the person leaves.

The walking time follows from the distance and the walking speed. In addition, we add noise to the walking time with a normal distribution whose spread is proportional to the walking time. For the search this means that the robot also misses a person in the right room if the person has not arrived yet or leaves the room during the room tour.

The stay time follows a distribution that depends on the room type. For most types it is a uniform distribution over a range that fits the type. For restrooms we use a lognormal distribution, because many short and few long stays happen there.

Matrix and stay times together decide where a person is at a given moment. A row of the matrix holds probabilities of a room change and not of a presence. We therefore do not derive the presence, we measure it. The simulation runs over one year and we sum for every person how long they stay in every room.

Figure 3 shows the result of that measurement. Note the bar of the workplace against all the others. We see that

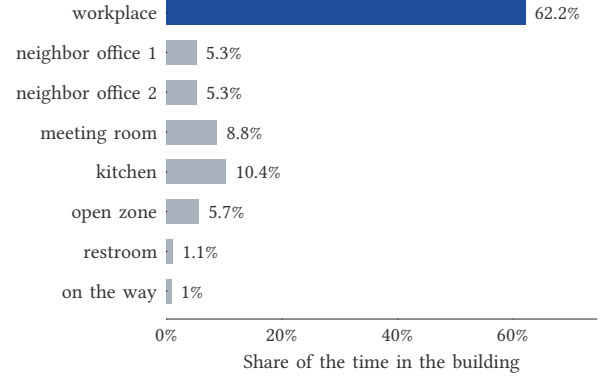


Figure 3. Where the people are, measured over one simulated year. Share of the time inside the building, summed over all people.

the desk alone takes almost two thirds of the time in the building, that the kitchen follows far behind with a tenth and that the restroom ends at one percent.

The measurement is also the oracle of Section VI.D. It is the true distribution of the model.

B. Precomputing the Room Tours

We determine the visibility points in advance for all rooms. The basis for this are the floor plan, the detection range and an entry point that stands for the access to the room.

We implement the room tour as an own event, which works through its points one after the other. At every point we check both ranges against the same precomputed visibility polygon of the viewpoint, which describes what the walls hide. Whether a person is only perceived or also identified is then decided by their distance alone.

VI. EVALUATION

We answer five questions in five separate experiments. For every experiment we first describe its setup and then evaluate it on its own.

A. Common Setup

All runs use the same building from Figure 1. Every person has one appointment per week, which gives 12 search jobs per day. Of every variant we compute 20 runs of 30 simulation days each, every run with a different seed. What a longer horizon changes is the subject of Section VII.B. We compare the variants on the same seeds, so every difference is paired per run. All intervals are 95 percent confidence intervals of the mean over the 20 runs, and in the text they follow the estimate in brackets.

The robot gets the time span $T_{\max}$ before the start of the appointment and has to find the person until then. A job counts as a failure if the robot does not find the person. The wanted person is present at the time of the search. The hit rate is the share of jobs with a find among all jobs.

The first two experiments set the boundary conditions, the time budget and the detection range. They compute with the full method and vary only this one setting. The three following experiments rate the method itself. Within one

experiment the variants differ in exactly one point, all other values follow Section IV.

B. Time Budget of the Search

The search starts a fixed span before the appointment, and this span is the time budget $T_{\max}$ from (2). The first experiment determines its size. We compute ten budgets from 2 to 20 minutes, and this at three detection ranges. The identification range r_i stays at 2 meters.

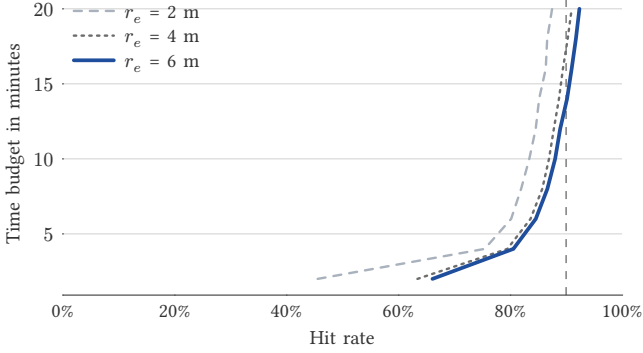


Figure 4. Time budget the robot needs for a given hit rate, at three detection ranges. The dashed line marks 90 percent, which the robot reaches after 14 minutes with a detection range of 6 meters.

Figure 4 shows the three ranges over the hit rate. Note where the curves cross the dashed line. We see that the robot reaches 90 percent after 14 minutes with a detection range of 6 meters and after 18 minutes with 4 meters, while with 2 meters it stays below the line over the whole axis.

A budget of two minutes is too short. The robot then reaches 66.1 percent, and for 7.8 percent of the jobs no room fits into the budget at all, so it does not start the search. At 4 minutes no rejections happen any more and the hit rate is 80.6 percent. After that every further minute brings only 0.7 percentage points.

The range shifts the whole curve. At a budget of 20 minutes 2 and 6 meters are still 4.9 percentage points apart, 87.5 against 92.4 percent.

For the other experiments we set the budget to 8 minutes, well past the bend. No jobs are rejected there any more, so what we measure is the quality of the method and not the shortage of time.

C. Perceiving at a Distance

The second experiment determines the detection range r_e . Four variants set it to 2, 4, 6 and 8 meters, while the identification range r_i stays at 2 meters. With the first variant $r_e = r_i$ therefore holds, so the robot only perceives people that it can identify anyway. The 8 meters are the physical limit of the cameras, so we check how much of this range the search needs.

The range works in two ways at once. The robot notices people from further away, and its room tour gets coarser. We therefore measure both effects together.

Figure 5 shows the four ranges. Note the step from 2 to 4 meters against the two steps after it. We see that the first

step is by far the largest and that the intervals of the last three points overlap each other.

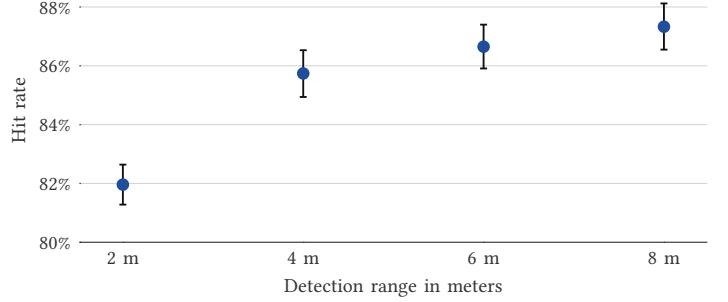


Figure 5. Four detection ranges over 30 days. The step from 2 to 4 meters carries almost the whole gain.

From 2 to 6 meters the hit rate rises from 82.0 to 86.7 percent, paired 4.7 percentage points (4.2 to 5.2). The full 8 meters add another 0.7 percentage points, but that interval covers zero. The last step therefore carries no weight.

For the three following experiments we set r_e to 6 meters.

D. Computing Versus Counting

The third experiment compares four strategies, in Figure 6 from the bottom up. The first gives every room the same value and serves as a lower reference point. It carries no information about the location, so in (3) only the cost decides and the robot drives to the room that takes the least time. The second uses the raw hit count h_i and the third the smoothed estimate $\hat{p}_i$.

The fourth strategy is an oracle. It hands the robot the rooms of a person together with the measured distribution of Figure 3, so the robot knows in advance how the person splits their day over these rooms. No estimate from observations can be better than this, so the oracle marks the ceiling of the search strategy. All four strategies use the same cost-aware site tour.

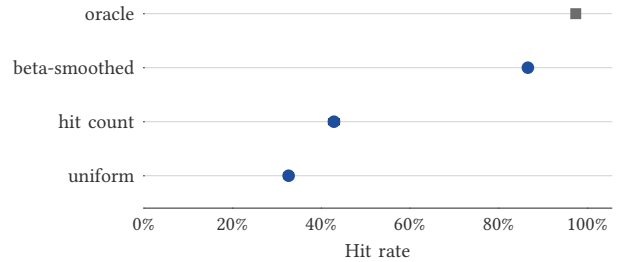


Figure 6. Four strategies over 30 days at $r_e = 6$ m. Counting alone already beats the uniform strategy, smoothing adds by far the largest step, and the oracle shows that the ceiling is not reached yet.

Figure 6 ranks the four strategies. Note the three steps between the four points. We see that all of them are clearly visible and that the step from the raw hit count to the smoothed estimate is by far the largest.

Counting alone already helps a lot. The uniform strategy reaches 32.7 percent and the raw hit count 42.9 percent.

Every observation that the robot makes on the way therefore pays off, even without any smoothing.

The smoothing adds a second step to 86.7 percent. It is worth 43.7 percentage points here and 52.2 at a short detection range, where the robot has fewer observations.

The oracle reaches 97.4 percent and thus stays clearly above the smoothed estimate, paired 10.8 percentage points (10.0 to 11.6). Over these 30 days the learned strategy is therefore far from its ceiling. Whether it stays there over a longer run is the subject of Section VII.B.

E. How Long the Prior Knowledge Helps

The fourth experiment runs at $r_e = 6$ m and varies the prior knowledge. The first variant gives every room the same prior value of 0.05. The second raises the workplace to 0.6 and leaves the other rooms at 0.05. Both variants weight the prior distribution with four assumed observations. The strategy is beta-smoothed in both variants.

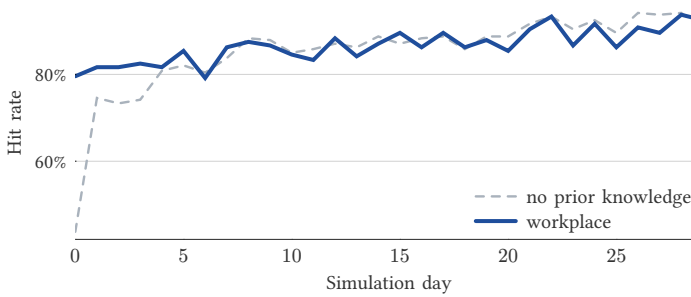


Figure 7. Hit rate over 30 simulation days at $r_e = 6$ m. The lead of the prior knowledge holds only for the first days, after that the two variants are indistinguishable.

The prior knowledge can only work as long as the robot has few observations of its own. We therefore evaluate the difference over the first days. The figure covers the whole experiment and thus also shows when the lead ends.

Figure 7 shows both variants over time. Note the left edge. We see that the two curves lie apart there and run indistinguishably after that.

Table I

Benefit of the prior knowledge per time budget, in percentage points against the same runs without it. On the first day it decides up to two fifths of the jobs, averaged over 30 days it disappears.

Budget	first day	30 days
4 min	+42.1	+4.9
8 min	+35.8	+1.3
12 min	+22.9	+0.3
20 min	+10.0	+0.2

On the first day the prior knowledge brings 35.9 percentage points, 79.6 against 43.7 percent. The following days bring much less.

The benefit also depends on the time budget. Table I compares the two variants at 4, 8, 12 and 20 minutes. Both columns give the hit rate with the workplace as prior knowledge minus the hit rate without it, once on the first

simulation day and once as the mean over 30 days. Note the right column in the two lower rows. We see that it falls to a few tenths of a percentage point, so from 12 minutes on the prior knowledge no longer makes a difference over the whole run.

With little time the prior knowledge therefore decides about the success, with plenty of time it does not matter. If the budget is enough for almost all rooms, the order plays no role, and then neither does its starting value.

F. Cost-Aware Site Tour

The fifth experiment compares two orders of the same rooms. The first sorts by the presence probability $\hat{p}_i$ alone, the second by the score w_i from (3) and therefore takes driving path and room tour length into account. At one detection range both variants drive the same precomputed room tours, so only the order of the rooms differs.

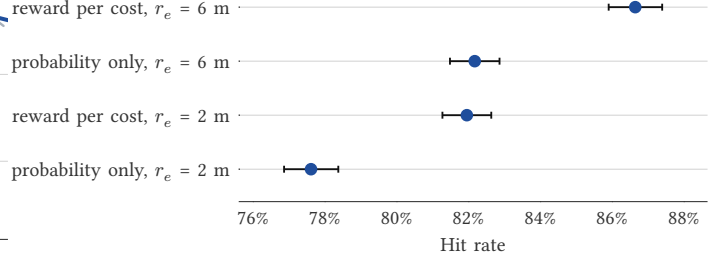


Figure 8. Two orders of the same rooms over 30 days at both detection ranges. The cost-aware order wins by about the same amount at either range.

At $r_e = 6$ m the cost-aware order reaches 86.7 percent against 82.2 percent, paired 4.5 percentage points (3.6 to 5.4). At a short range it is 4.3 percentage points (3.5 to 5.1).

Figure 8 shows both orders at both ranges. Note the two pairs against each other. We see that the distance within a pair is about the same above and below, so the cost-aware order does not depend on how far the robot perceives.

VII. DISCUSSION

A. Interpretation of the Results

Three of the five experiments show three ways to make up for the same lack of observations. The smoothed estimate, the prior knowledge and the larger detection range all help when the robot has too few observations of its own.

That is a good result, because the three levers can stand in for each other. A robot with a short detection range makes up the gap over smoothing and prior knowledge, and a careful prior knowledge pays off above all when the range stays small.

The same cause lies behind the dependence on the budget from Table I. Whoever has enough time drives to almost all candidates anyway, and then the order does not matter. The time budget is therefore not a side parameter, it also decides whether a measure works at all.

The cost-aware site tour works differently. It does not save observations, it saves time, and it works at both ranges. At

a short range it turns out a bit stronger, because the longer room tours bind more time there. The measure adds up to the others.

B. What a Longer Run Changes

All experiments above run over 30 simulated days, and that window is the steep part of the learning curve. A separate run over one simulated year, identical in everything else, shows what comes after it. The smoothed estimate keeps improving for a few months, then stops, and it ends up level with the oracle. The gap of Section VI.D therefore belongs to the first weeks of a deployment and not to the method.

C. Temporal Patterns

We have left the time dependence out on purpose. The semi-Markov chain from Section IV does not depend on time, because the target room follows from the current room alone. The model of the people therefore knows no time of day.

Still, the extension is worth a try, and the starting point would be the estimate. Every observation enters the same counter with the same weight, no matter when it arises, so an observation from the morning moves the estimate for the afternoon. With a time axis it could be assigned to the matching time window instead. Krajník et al. modeled the probability as a function of time and lowered the median of the search time by 35 to 65 percent [5]. A related idea lets old sightings lose weight, so that the estimate follows a person who changes their habits.

D. Limits of the Model

A part of the failures is structural and open to no search strategy. A small part of the jobs meets a person in a restroom, which we exclude from the search. In addition, there are failures where the person changes the room during the search or is just on the way. The smoothed estimate reaches 86.7 percent and the oracle 97.4 percent, so the rest up to a hundred percent does not go back to the search strategy alone.

One idea uses the time between the jobs. Today the robot gathers sightings only while it searches. It could instead use its idle time to collect them on purpose, for example by watching a room for a while. Its model would then grow without a job being needed.

E. Relation to the Related Work

The coupling of both levels delivers above all one thing. A miss becomes usable, because the robot has provably searched the room completely. With Mehdi and Berns this guarantee is missing. The authors state themselves that the robot can miss the person in the right room [6]. With Krajník et al. the question does not arise, because a node has no inner geometry [5].

The value of the complete search does not lie in the amount of data, it lies in the fact that a miss may count at all. Without this guarantee the robot would have to drop every unsuccessful visit and would learn from hits only.

Unlike in both works, the wanted person keeps moving during the search in our case and stays outside all searchable

rooms from time to time. That creates failures that no better model could prevent. Even with a good model the hit rate stays clearly below a hundred percent.

VIII. CONCLUSION

We have solved the person search in a building with a site tour and a room tour. For the site tour the robot estimates a smoothed presence probability for every room and orders them under the time budget. On the room tour it covers the room completely. We have evaluated this in a discrete event system over 30 days and 20 runs per variant.

The initial question can now be answered. To find a person in 90 percent of the jobs, the robot has to start 14 minutes before the appointment. With a budget of 8 minutes it reaches 86.7 percent, against 32.7 percent with a uniform value for all rooms. Given a year of observations it reaches the level of an oracle that knows the model.

The measures towards this goal work in different ways. The smoothed estimate, the prior knowledge and the larger detection range all attack the same cause, too few observations. If the perception gets better, the gain of the smoothing shrinks and the gain of the prior knowledge disappears. The cost-aware site tour works independently of this and at every range.

REFERENCES

- [1] C. G. Cassandras and S. LaFortune, *Introduction to Discrete Event Systems*. Cham: Springer International Publishing, 2021. doi: 10.1007/978-3-030-72274-6.
- [2] S. Macenski, F. Martin, R. White, and J. G. Clavero, "The Marathon 2: A Navigation System," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2020, pp. 2718–2725. doi: 10.1109/IROS45743.2020.9341207.
- [3] "Planner Server — Nav2 1.0.0 documentation." Accessed: Aug. 05, 2026. [Online]. Available: <https://docs.nav2.org/configuration/packages/configuring-planner-server.html>
- [4] R. IMVS, "Tech-Talk: Vertex Visibility Algorithm of a simple Polygon." Accessed: Aug. 05, 2026. [Online]. Available: <https://www.fhnw.ch/plattformen/imvs/tech-talk-vertex-visibility-algorithm-of-a-simple-polygon/>
- [5] T. Krajník, M. Kulich, L. Mudrova, R. Ambrus, and T. Duckett, "Where's waldo at time t ? using spatio-temporal models for mobile robot search," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, Seattle, WA, USA: IEEE, May 2015, pp. 2140–2146. doi: 10.1109/ICRA.2015.7139481.
- [6] S. A. Mehdi and K. Berns, "Behavior-based search of human by an autonomous indoor mobile robot in simulation," *Universal Access in the Information Society*, vol. 13, no. 1, pp. 45–58, Mar. 2014. doi: 10.1007/s10209-013-0301-8.
- [7] A. Sarmiento, R. Murrieta-Cid, and S. Hutchinson, "A Multi-robot Strategy for Rapidly Searching a Polygonal Environment," *Advances in Artificial Intelligence – IBERAMIA 2004*, vol. 3315. Springer Berlin Heidelberg, pp. 484–493, 2004. doi: 10.1007/978-3-540-30498-2_48.
- [8] B. L. Golden, L. Levy, and R. Vohra, "The orienteering problem," *Naval Research Logistics*, vol. 34, no. 3, pp. 307–318, Jun. 1987. doi: 10.1002/1520-6750(198706)34:3<307::AID-NAV3220340302>3.0.CO;2-D.
- [9] P. Vansteenwegen, W. Souffriau, and D. V. Oudheusden, "The orienteering problem: A survey," *European Journal of Operational Research*, vol. 209, no. 1, pp. 1–10, Feb. 2011. doi: 10.1016/j.ejor.2010.03.045.
- [10] A. A. Johnson, M. Q. Ott, and M. Dogucu, *Bayes Rules! An Introduction to Applied Bayesian Modeling*. CRC Press, 2022.